Python Technical Documentation

This document provides a technical overview of the Python source code developed for the Reconfigurable Integrated Circuit Test Fixture project. It explores the key design decisions and notable data structures that are the foundation of the codebase, as well as how the software interacts both internally and with the associated hardware and firmware components. In addition, the document outlines considerations for future development, offering guidance on future expansion while preserving the program's modularity, clarity, and overall organization.

High Level Overview

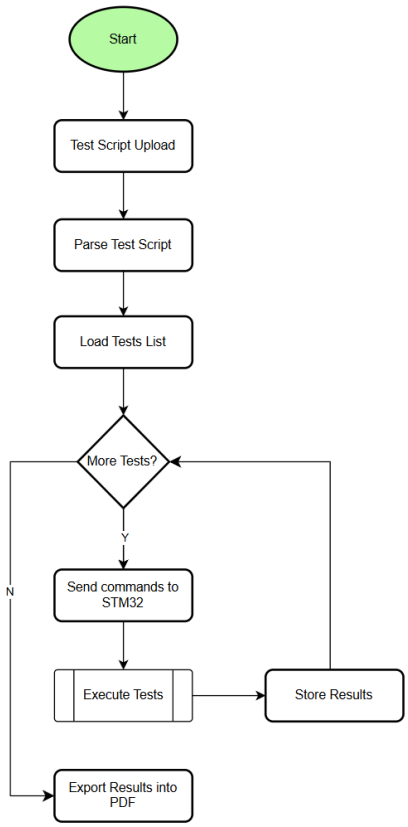

Figure 1. High level flowchart of program during runtime

The application lets users upload a test script, execute the defined tests, and generate a PDF report of the results. As shown in the flowchart, the program follows a straightforward linear sequence. This approach keeps execution predictable and debugging manageable. Each stage depends on the successful completion of the previous one. Any errors halt the process early and prevent downstream steps from running.

The workflow begins with uploading a test script. Next, the application parses its contents and loads the extracted tests into a list. It then enters a loop to check whether additional tests remain.

If so, commands are sent to the STM32 microcontroller, the tests are executed, and the results are stored. The process returns to the decision point until all tests are complete. Finally, the accumulated results are exported into a PDF report.
Most of these operations are handled within the main GUI event loop. The only asynchronous components are the transmission of commands to the STM32 and the storage of test results. While this design generally maintains responsiveness, the GUI can become temporarily unresponsive when handling large or complex scripts—particularly during the upload and parsing stages, where the interface attempts to render and process all script lines at once.

Module	Description	File Name	File Description
device	Handles communication to hardware and require data structures to read and write information	test_runner.py	Executes parsed test scripts asynchronously from TestVector objects
		test_vector.py	Implements TestVector object to stored parsed information from test scripts and results received during test script execution
file_io	Handles all file-related operations	parser.py	Parsing for algorithm test scripts using PyYAML
		report.py	PDF report generator using ReportLab's platypus module
gui	Construct GUI elements and connect backend code in device and file_io	main_window.py	Acts as top-level, constructs the main window of the GUI using PySide6 and integrates all portion of the Python code together
		tabbed_editor.py	Implements tabbed text editor widget, like NotePad++, VSCode, etc.
		test_script_wizard.py	Implements QWizard widget that assist in creation of test scripts by offering high-level support in the form of dropdown menus.

Table 1. Source code layout with brief description

The application contains three modules: device, file_io, and gui, as displayed in Table 1.

The device module implements asynchronous communication with the virtual COM port of the IC test fixture to prevent blocking the main GUI event loop. The module also implements the data structures required to store parsed information from test scripts and the results received. This module is used in the “Parse Test Script” step and the test script execution loop in the flowchart.

The file_io module handles all related file operations, including the implementation of the parsing algorithm, and exporting the results as a PDF file after executing the test script. The module is responsible for the “Parse Test Script” and “Export Results to PDF” steps of the flowchart.

The gui module creates the GUI, including the main window, and user-interactable components required to execute all operations from the flowcharts. These components are connected to the backend code from the device and file_io modules.

File IO Module

Parser

This file implements the parsing algorithm used to parse the test scripts. The formatting and expected syntax for the test scripts can be found in docs/YAML Test Script Guide. The low-level parsing is done by using PyYAML. PyYAML will read the test script and create a dictionary with keys and values being proper Python data types. The parsing algorithm manually checks that the data is within the hardware’s capabilities and is a valid argument.

At the top of the file are the global macros used by the parsing algorithm. These variables provide the expected logic, voltage levels, and maximum pins the hardware can support. They are implemented as sets for constant lookup times.

Beneath the global macros are parser exceptions. All parser exception types inherit from Python’s base “Exception” class and have no implementation themselves. The purpose of these exceptions is to display the exception type and its error message to the user in the GUI. This allows users to identify the location of the error. For example, a “MissingKeys” error will be shown to the user as “MissingKeys: error_msg.” Adding a line number would also be useful; however, PyYAML does not maintain line numbers for reference.

Following the parser exceptions are the helper functions, denoted by their naming scheme “check_parameter.” These helper functions are used throughout the parsing algorithm, since many operations are repetitive, to ensure arguments are valid. They raise an error if the argument is invalid and simply return if they are.

The functions responsible for the actual parsing are denoted by “parse_section.” These functions are implemented in the same order they are called. All functions first check if the expected

sections/parameters are in the test script; otherwise, an error is raised. Any unknown keys will raise a warning.

Inside the “parse” function, PyYAML performs all the low-level parsing, creating a dictionary with all values converted to valid data types, such as int, float, string, or list. Since the data is a dictionary, parsing can easily be accomplished by directly accessing parameters by their designated key in the test script. For example, data[“Global Parameters”][“VCC Pin”] accesses the VCC Pin. Then the VCC Pin can be determined if the pin value provided is valid. There is a function dedicated to parsing each section of the test script. This is done for modularity if more sections are added to the test script in the future. The current parsing order is “Global Parameters,” “Pin Map,” “Truth Table,” and “Tests.”

The functions “parse_truth_table,” and “parse_tests” are unique in that they also parse their respective sections into a different data structure. For the “Truth Table” section, PyYAML creates a list nested dictionary, the “parse_truth_table” function will transform the nested dictionaries shown in Figure 2 into a single dictionary where the column names of the truth table are the keys, and all the values from the nested dictionaries are concatenated into a single list as shown in Figure 3.

```
Truth Table:
  - {A: L, B: L, Y: H}
  - {A: L, B: H, Y: H}
  - {A: H, B: L, Y: H}
  - {A: H, B: H, Y: L}
```

Figure 2. Data structure of Truth Table section in test scripts

```
Truth Table:
  A: [L, L, H, H]
  B: [L, H, L, H]
  Y: [H, H, H, L]
```

Figure 3. Resulting data structure of transformed dictionary

The reason why the formatting in Figure 3 of the test scripts isn’t used instead of the structure in Figure 2 is to allow users to write truth tables in the test scripts resembling how truth tables are typically represented. Truth tables typically have inputs on the leftmost columns, and outputs on the rightmost columns, each row representing the inputs required to get the output. From Figure

3, the truth table appears to be rotated, where the rows have the names of the columns, and the columns represent the row of the truth table.

The “parse_tests” function creates a list of TestVector objects. Any information that is required for testing should be passed as argument to this function. The helper function “parse_test_IO” parses the “Inputs” and “Outputs” section of each individual test. In this function, each key-value pair is wrapped into an IOCommand data class. The voltage type and command type are determined by the written command of the test script.

Report

This file is responsible for exporting the data inside TestVector objects, and other information inside the test script, and exporting the results as a readable PDF file. All data is exported in tabular format for dynamic expansion and formatting. PDF reports are generated using the platypus module from the ReportLab project. The module contains high-level abstractions for creating PDF documents, drawing objects, auto-formatting, and sizing.

At the top of the file are constants for drawing T-charts for the sections “Chip Info,” “Pin Map,” and “Global Parameters” sections of the test script. The data will simply be represented as key-value pairs; the left side of the chart contains the keys, and the right side of the chart contains the values. The “dict_to_table” function is responsible for creating these types of tables. The “Truth Table” section is not included as the data from the TestVector objects contains all the inputs, expected outputs, and results to be printed out in a tabular format.

The “export_to_pdf” function generates the contents and builds the PDF report. The list variable story is all of the contents to be displayed in the PDF report. The platypus module will display the contents in the same order as the story array, meaning the first element will be at the top of the first page, and the last element will be at the bottom of the last page. The function first generates the T-charts by calling “dict_to_table” and appends them to the story. Then the function will iterate over a list of TestVector objects, calling the “export_to_table” function. For each TestVector object, it will create a table for the data and create a TableStyle object to format the table using the metadata returned from the “export_to_table” function. This is accomplished by creating style commands and concatenating all of them into a list. The primary purpose of these commands is mainly to center and span empty cells in the table. Empty cells are equivalent to empty strings, and spanning will combine cells together. Spanning is useful for creating cells that need to be subdivided into multiple rows or multiple columns.

Device Module

Test Vector

This file implements the TestVector class and all associated data structures required for testing.

The enumerated type LogicMapping is used to represent the type of command. The type of command determines how the specified pin(s) map to the specified value(s).

The data class IOCommand represents a line in the “Inputs” and “Outputs” sections of tests. IOCommand stores the pins, and their respective values are stored as a list. The fields volt_type and cmd_type are used during testing for the voltage of a HIGH signal, and how to map pins to their respective values.

The data class PinResult is used for storing the received logic and ADC result (in Volts).

The data class Condition is used for representing a test condition. The possible conditions are the VCC voltage and its respective output low and high thresholds from the test script. This is used by TestRunner to keep track of the current condition a test is being performed with.

The TestVector class contains all the necessary information used to perform a test. Any additional information needed should be added to the constructor. TestVector contains the following: global_params, pin_map, inputs, outputs, results, test_name, and passed. The first two fields contain information for test conditions and the abstracted pin names. The inputs and outputs are a list of IOCommands that are used to write commands and compare results, respectively. The results field is a 3-layer dictionary used to store the results received from the test fixture. The remaining fields are for the name of the test provided in the test script, and a Boolean for if a test has passed, and all results match the expected outputs. If an input or output contains a serial command, all inputs and outputs are padded to be the same length as the longest serial command. The padded logic is the last logic value in the command. This is because the firmware expects serial commands to be the same length, or the logic is a single voltage.

The results field is a 3-layer dictionary, so each result for a pin can be accessed in constant time. The keys for the layers of the dictionary are in this order: the VCC voltage condition, the step number, and the pin that was measured. The step number is an output of the firmware, which is used for serial inputs. For tests with no serial inputs, the step number is always 0.

The methods “test_conditions” and “power_pins” return all test conditions and power pins, including the VCC voltage from global_params. These are used for generating the PRM and VIN commands expected by the firmware. The method “pin_lists” converts all input IOCommand into a list of pins and voltage input, and all output IOCommand to a list of pins to be measured by hardware. These lists are used for generating the INS, VIP, and OUT commands.

The method “compare_results” will compare all output IOCommand to the logic received from the test fixture stored in results. In this function, the logic will be converted from an integer to the same value type used in the test script. For example, the firmware returns a 0 for LOW, if the expected output for the pin is “L,” 0 is replaced with “L” inside the results. If the expected output was an integer, then nothing changes. When exporting the results to a PDF report, the expected output and received results share consistent formatting. If the received logic does not match the expected output, TestVector’s pass field is set to False and cannot be changed.

The method “export_to_table” exports all the IOCommand data classes and results in a tabular format, along with their metadata. The exported table is a 2D array, where each list in the array represents a row of data, which is represented by strings. The table includes headers and empty cells (created by empty strings, “”). The empty cells are used when the data in cells needs to span multiple rows or columns. It does not matter where the data is for spanning, but it is recommended to be stored in the left and top-most cells, as the metadata of the table assumes this logic. Unlike input pins that may share columns, the output pins are unraveled from the lists and receive their own columns. This allows the results of each pin to have their own cell to enhance the readability of the table. If a test has multiple test conditions, the VCC column is included, it is omitted. If the VCC column is included, the expected result cells are subdivided to match the number of different test conditions.

The metadata of the table is used for formatting the table in the PDF report, specifically indicating to ReportLab Platypus to span certain rows and columns. The metadata dictionary includes input_span, output_span, num_rows, include_vcc, and num_vcc. The input_span represents the number of input IOCommand. The output_span represents the number of output pins in total from all output IOCommand. These two parameters affect the spanning for rows. The remaining parameters represent the number of rows in the table, if the table includes a VCC column, and the number of different test conditions. These arguments affect the column and row spanning.

The private methods, denoted by the starting “_” character, are helper functions used to assist in translating higher-level test script inputs into low-level hardware commands, or vice versa for test script outputs. These functions should never be called directly outside of unit testing.

For debugging purposes, the method “simulated_test,” and its helper “_get_exp_val,” will roughly simulate the execution of the test. These methods are used in unit tests to verify the implementation of the “_compare” and “export_to_table” methods.

Test Runner

This file implements the TestRunner class. TestRunner executes tests in the given list of TestVector objects at all specified VCC voltages. All commands are sent encoded in UTF-8, with a new line at the end of each string. The implementation assumes that after every command is received, the STM32 will send a response ending with a new line character. This is important because commands are sent directly after receiving a response.

The TestRunner fields cmds and buffer are used for writing and reading responses. The field cmds is a deque, which stores all the commands of a test at a condition. The first element of the deque is the command sent to the hardware. If there are no more elements in the deque, nothing happens. The field buffer stores all the responses from the hardware as a string. The remaining fields, conditions, test_idx, cond_idx, and stop, are used to keep track of the current progress of the tests. If stop is True, an error occurred during testing, and no commands will be sent.

TestRunner includes 2 signals: done, error, and status_msg. The signal done emits when all tests have been successfully executed, error emits if an error occurred during testing, and status_msg emits updates in the form of strings. These updates are simply commands sent and responses received. All these signals are connected to the main window of the GUI to update the progress of the test execution.

TestRunner runs asynchronously to the GUI; this is done by connecting the QSerialPort's readyRead signal to the "handle_ready_read" method in TestRunner. This method decodes the response into a string and adds it to the buffer. While there is a newline character in the buffer, it will process each line appropriately based on the response. Currently, the expected responses are based on the starting characters: "ERR," "Done," and "STEP." If an "ERR" is received, the error signal is emitted, and test execution is stopped. If "Done" is received, this means the test executed successfully, and if "STEP" is received, the string is parsed, and the resulting ADC value and logic are stored into the current TestVector object. Any other response is simply printed to the GUI, and the next command is sent if possible.

GUI Module

Main Window

This file serves as the top-level entity, connecting all backend code in the device and file_io modules with GUI elements.

All GUI elements are created using PySide6, a Python binding to the Qt framework. Qt offers additional apps such as QtCreator, an IDE for GUI development. QtCreator offers a drag-and-drop editor, which is useful for creating complex GUI elements if programming them in Python is difficult. The PySide6 library is used for this project due to its modern UI appearance and wide widget selection, including QSerialPort, which runs asynchronously, simplifying the communication portion. PySide6 also supports signals and slots that can be used to execute tasks outside of the GUI event loop.

The main window offers basic text editor support, like NotePad++ and Visual Studio Code. This allows users to edit test scripts with minor mistakes without needing another text editor app. The QTextEdit widget at the bottom displays updates throughout the program, mainly serial communication with the hardware to inform users of any errors the program encounters and update the status on test script execution. This also acts as a log for the program.

This file will create instances of any objects that are required during the application's runtime and include any default settings. Currently, it dynamically finds the serial port by searching for the CP2102 USB-to-UART bridge information (VID: 0x10C4, PID: 0xEA60), with the serial number 554D4C2043617073746F6E652032352D333034204E5557432032303236.

Tabbed Editor

This file creates the Tabbed Editor widget, a collection of QPlainTextEdit objects switchable via the QTabWidget. Most basic text editor features are implemented here, such as key shortcuts, special symbol on the tab, indicating the contents of QPlainTextEdit object has been modified. All functions are implemented based on the current QPlainTexEdit in focus on the GUI as it is unlikely a user will want to modify a file that is not in focus.

A dictionary is used to keep track of which file each QPlainTextEdit object originates from. If a blank QPlainTextEdit was created, the file path is None. The dictionary's key is the QPlainTextEdit object, and the file path is the value. When a tab is closed, the QPlainTextEdit is popped from the dictionary and the QPlainTextEdit is saved before deleting.

Tabbed Editor has two signals tab_added, and no_tabs. These two signals are connected to the main window of the GUI. The main window will display the directions to create a new file when there are no tabs opens, and swap to the tabbed editor widget when there is at least one.

Test Script Wizard

This file creates the TestScriptWizard widget, a QWizard widget offering high-level support for YAML test script generation. The widget simplifies the test script writing process by using a variety of dropdown menus supported by PySide6. The options for these dropdown menus are limited based on the macros declared in parser.py to guarantee the generated test script is valid.

At the top of the file are sorted lists of the selectable options. These sorted lists are used to instantiate QComboBox widgets, or simply dropdown menus. The "get_value" function will extract the value from any widget. This function can easily be expanded should more types of widgets be used.

Every page of the widget derives from the QWizardPage object. Each page implements its own "validatePage" function. The function returns a Boolean; if false, it will prevent the user from moving onto the next page. The function will extract the data from the entries the user provided into the expected structure that the parsing algorithm expects. The page's respective "parse" function is called for validation. The order of the pages is the same order as the parsing algorithm's order. To share information between the pages, as some parsing functions are dependent on other portions, TestScriptWizard contains a dictionary that is accessible via wizard().data in the pages. The page numbers are located near the top of the file as an enumerated type named PageNum. PageNum allows page numbers to be easily changed if the parsing order changes. Page numbers start from 0 and increase upwards. A page number of -1 means it is the last page. The last page (currently TestsPage) will use PyYAML to generate the .yaml file before closing Test Script Wizard.

Most sections of the test scripts can grow dynamically as they are designed to be capable of adapting to the constraints of the chip under test. For example, the "Pin Map" section is the name of the pin, and the pin number on the socket maps to. The number of pins can range from 0 (omitted) to 18 (VCC and GND Pins are reserved). To not flood the screen with dropdown

menus or other widget types, the DynamicContainer class is used to create a grid-like arrangement of QWidgets. DynamicContainer grows dynamically based on the number of entries the user requires. DynamicContainer stores these rows as a list of tuples. The first element of the tuple is the list of widgets in the row, and the second element is the delete button for that row. During deletion of the row, the index of the row is located, and the element at the list is popped all widgets in the tuple are deleted. Then, all remaining rows are shifted upwards.

Unit Tests

The unit tests only test the parsing algorithm in parser.py, and the TestVector class in test_vector.py. The parsing algorithm and TestVector class are the most crucial as they serve as the bridge between the abstractions in the test scripts and sending and receiving commands to the hardware portion of the IC Test Fixture. Unit tests are written with the support of the PyTest library. It is highly recommended to run these unit tests after making changes to test scripts or source code to ensure everything works as expected.

Test classes organize related tests together; for example, the parsing functions have many different conditions where errors are raised. Additionally, using the "::" operator allows developers to run a specific group of tests if it's unnecessary to run all of the unit tests. The two primary decorators used from the PyTest library are "pytest.fixture," and "pytest.mark.parametrize." The first decorator creates a reusable test fixture that all tests in the file can use. Each test will have a unique copy of the test fixture, avoiding any issues where data may be modified in between tests. The second decorator allows the parametrization of test functions, meaning the same function can be used with different inputs. This reduces the amount of code written to do similar tests with only minor changes to the inputs, such as edge cases.

Note: PyTest will automatically find test functions and test classes, but they must start with "test" and "Test" respectively. Without those keywords, PyTest will assume it is not a test and skip it.

Future Expansion

Coding Style

Any future expansion to the code base should be written following the PEP8 style guide for Python (https://peps.python.org/pep-0008/). The guide outlines standard Python coding conventions that are commonly used for small hobbyists and industry projects.

For documenting the code, it is recommended to use Google style docstrings (https://gist.github.com/redlotus/3bc387c2591e3e908c9b63b97b11d24e). The link provides an example of writing docstrings for various types of code. Many commonly used IDEs support this type of docstring. When hovering over variables, function names, classes, etc, developers will be able to read a short summary of its purpose and expectations if applicable.

Code Expansion

All code should be written with the expectation that it is executed from the software/ directory. This is especially important for any file paths that may are used, whether for configuration or testing. Additionally, to enable cross-platform development, it is recommended to use the pathlib library, as it is OS-independent and will automatically create file paths expected by the OS.

To maintain the project's structure, it is important to create any new source files inside a module, whether it's an existing one listed in Table 1.1, or a new module (or submodule) if the source code implements a distinct new feature. The __init__.py file indicates to the Python interpreter that a directory is a package; while not required, it is still highly recommended for modern Python projects. The main.py file should likely not be changed unless the GUI color palette is desired. This file serves as the user entry point of the application and should never implement any core logic or features.

If the test script is expanded by adding a new section, a new function named "parse_section_name" should be implemented in parser.py. For any additional subsections to an already existing section in a test script, the parsing logic should be added to the parsing function for that section or abstracted into a helper function that is called by the parsing function. This maintains the organization of the code, where each section of the test script has its own parsing function and relative helpers. New sections that need to be displayed in the PDF report either need to be a part of the return value for the "parse" function or added as a field in the TestVector class, which is created in the "parse_tests" function.

Any serial communication must be handled asynchronously to prevent the GUI from freezing. The main method for this is PySide6 Signal, which emits a signal under certain conditions and can be connected to a function that will be automatically executed. This prevents the need to poll for potential inputs (typically done by waiting in while-loops), which halts the main event loop. Another method is creating a separate thread using QThread. PySide6 documentation recommends using the worker approach, where a QThread is wrapped inside a QObject to separate the logic you want to run from the responsibilities of QThread. More can be read here: https://doc.qt.io/qtforpython-6/PySide6/QtCore/QThread.html. Creating new threads is only necessary if multithreading performance is required. This is when operations are either expensive or take a long time to execute. Currently, the application only freezes when handling large test scripts, but since most test scripts are expected to be small, QThread objects were not used.

All new code should somehow be connected to main_window.py to ensure it is a part of the GUI. Ideally, all parts of the code should also display some status message to the GUI to let the user know of the processes happening, and act as pseudo-log.

Unit tests located in the tests/ directory should be run every time changes are made to either parser.py or test_vector.py. These tests should also be updated if new functions are added or logic is changed. New YAML files should be added to the unittest_yaml/ directory to test various incorrect and correct test scripts.

Package API

Package	Documentation
PySide6	https://doc.qt.io/qtforpython-6/index.html
PyYAML	https://pyyaml.org/wiki/PyYAMLDocumentation
ReportLab	https://docs.reportlab.com/reportlab/userguide/ch1_intro/
PyTest	https://docs.pytest.org/en/stable/index.html
PyInstaller	https://pyinstaller.org/en/stable/

Table 2. Third-party packages used and links to their documentation.